

# **RISK-BASED FRAMEWORK FOR ADAPTIVE QUALITY OF SERVICE AND THREAT-AWARE RESPONSE IN SOFTWARE- DEFINED NETWORKING**

K.C.S.P.Perera

(IT22324306)

BSc. (Hons) Degree in Information Technology Specialized in Computer Systems  
and Network Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

June 2025

## DECLARATION OF THE CANDIDATE & SUPERVISOR

I declare that this is our own work, and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or Institute of higher learning, and to the best of our knowledge and belief, it does not contain any material previously published or written by another person except where the acknowledgment is made in the text. Also, I hereby grant to the Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other mediums.

| Name           | Student ID | Signature                                                                            |
|----------------|------------|--------------------------------------------------------------------------------------|
| K.C.S.P.Perera | IT22324306 |  |

The supervisor/s should certify the proposal report with the following declaration.

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

- Mr. Uditha Dharmakeerth (Supervisor)
- **Ms. Shashika Lokuliyana** Arachchi (Co-supervisor)

.....

Signature of the supervisor

.....

Date

## ACKNOWLEDGEMENT

Throughout the development of this research on the **Machine Learning-Based Intrusion Detection and Threat Scoring Component for Software-Defined Networking (SDN)**, I would like to express my sincere gratitude to all individuals who supported and guided me in successfully completing this work.

First and foremost, I would like to express my deepest appreciation to my dissertation supervisor, **Mr. Uditha Dharmakeerth**, for his invaluable guidance, continuous support, and constructive feedback throughout the research process. His insights and direction were instrumental in shaping this component and ensuring its successful integration within the overall research framework.

I would also like to extend my gratitude to the **Department of Computer Systems and Network Engineering** and the academic staff for providing the necessary knowledge, resources, and technical support. Their guidance in areas such as **network security, machine learning, and SDN architecture** played a crucial role in the successful development of this system.

Special thanks are extended to all lecturers and course coordinators whose teaching throughout my academic journey helped build a strong foundation in **networking concepts, data analysis, and machine learning techniques**, which were essential for this research.

I would also like to acknowledge the developers and contributors of the **open-source tools and technologies** used in this project, including **Python, Scikit-learn, FastAPI, and data processing libraries**. These tools were fundamental in implementing the intrusion detection model and the threat scoring mechanism.

Finally, I would like to express my heartfelt gratitude to my **group members** for their collaboration and support in developing the overall SDN-based system, as well as to my **family and friends** for their encouragement and motivation throughout this journey. Their support has been invaluable in completing this research successfully.

## ABSTRACT

The increasing complexity of modern network infrastructures has introduced significant challenges in maintaining both security and performance. Traditional intrusion detection systems are largely rule-based and lack the ability to detect evolving or unknown cyber threats in real time. To address these limitations, this research presents a **machine learning-based intrusion detection and threat scoring component** within a broader **Software-Defined Networking (SDN) framework** for intelligent network management.

The proposed system utilizes a **flow-based traffic analysis approach**, where network packets are aggregated into flows and represented using statistical features such as packet counts, flow duration, and inter-arrival times. A **Random Forest classifier** is trained using the CICIDS2017 dataset to accurately distinguish between benign and malicious traffic. To enhance decision-making, the model incorporates a **probability-based threat scoring mechanism**, producing continuous risk scores instead of binary classifications. Probability calibration and threshold optimization are applied to improve reliability and minimize false negatives.

The system is implemented using a modular architecture, where the trained model is deployed as an API service and integrated with an SDN controller. This enables real-time or near real-time analysis and supports adaptive policy enforcement based on threat levels.

Experimental results demonstrate strong classification performance, with high ROC-AUC and PR-AUC values, confirming the model's effectiveness in handling imbalanced network data. Overall, the research highlights the potential of combining **machine learning, flow-based analysis, and SDN integration** to develop a scalable and intelligent network security solution.

**Keywords:** Intrusion Detection System, Software-Defined Networking, Machine Learning, Threat Scoring, Flow-Based Analysis, Random Forest.

## LIST OF ABBREVIATIONS

| Abbreviation | Meaning                              |
|--------------|--------------------------------------|
| SDN          | Software-Defined Networking          |
| ML           | Machine Learning                     |
| IDS          | Intrusion Detection System           |
| NIDS         | Network Intrusion Detection System   |
| QoS          | Quality of Service                   |
| API          | Application Programming Interface    |
| RF           | Random Forest                        |
| SVM          | Support Vector Machine               |
| DDoS         | Distributed Denial of Service        |
| DoS          | Denial of Service                    |
| TCP          | Transmission Control Protocol        |
| IP           | Internet Protocol                    |
| CSV          | Comma-Separated Values               |
| JSON         | JavaScript Object Notation           |
| PKL          | Pickle File (Serialized Model File)  |
| ROC          | Receiver Operating Characteristic    |
| AUC          | Area Under the Curve                 |
| PR           | Precision-Recall                     |
| FPR          | False Positive Rate                  |
| TPR          | True Positive Rate                   |
| TP           | True Positive                        |
| TN           | True Negative                        |
| FP           | False Positive                       |
| FN           | False Negative                       |
| IAT          | Inter-Arrival Time                   |
| OVS          | Open vSwitch                         |
| NVD          | National Vulnerability Database      |
| CVE          | Common Vulnerabilities and Exposures |
| CVSS         | Common Vulnerability Scoring System  |
| REST         | Representational State Transfer      |

## Table of Contents

|                                                         |     |
|---------------------------------------------------------|-----|
| DECLARATION OF THE CANDIDATE & SUPERVISOR .....         | i   |
| ACKNOWLEDGEMENT .....                                   | ii  |
| ABSTRACT .....                                          | iii |
| List of abbreviations.....                              | iv  |
| 1.INTRODUCTION.....                                     | 1   |
| 1.1 Background .....                                    | 1   |
| 1.2 Background & Literature Review .....                | 3   |
| 1.3 Research Gap .....                                  | 6   |
| 1.4 Research Problem .....                              | 7   |
| 1.5 Research Objectives.....                            | 8   |
| 2. METHODOLOGY .....                                    | 8   |
| 2.1 System Requirements .....                           | 8   |
| 2.1.1 Functional Requirements.....                      | 9   |
| 2.1.2 Non-Functional Requirements.....                  | 10  |
| 2.2 System Design .....                                 | 11  |
| 2.2.1 Overall System Architecture .....                 | 11  |
| 2.2.2 Component Interaction Diagram .....               | 13  |
| 2.3 Data Collection.....                                | 14  |
| 2.4 Data Preprocessing .....                            | 16  |
| 2.5 Machine Learning Model Development .....            | 18  |
| 2.5.1 Model Selection .....                             | 18  |
| 2.5.2 Dataset Preparation and Splitting.....            | 18  |
| 2.5.3 Model Training.....                               | 19  |
| 2.5.4 Probability-Based Threat Scoring.....             | 19  |
| 2.5.5 Threshold Optimization .....                      | 20  |
| 2.5.6 Model Evaluation .....                            | 20  |
| 2.5.7 Model Deployment Considerations .....             | 21  |
| 2.6 Implementation (Tools, APIs, SDN Integration) ..... | 21  |
| 2.6.1 Programming Environment and Tools .....           | 21  |
| 2.6.2 Machine Learning Model Integration .....          | 22  |
| 2.6.3 API-Based Inference Module .....                  | 22  |

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 2.6.4 SDN Environment and Integration .....                         | 22 |
| 2.6.5 System Workflow Implementation .....                          | 23 |
| 2.6.6 Configuration and Flexibility .....                           | 24 |
| 2.7 Testing.....                                                    | 24 |
| 2.7.1 Testing Strategy .....                                        | 24 |
| 2.7.2 Model Performance Testing .....                               | 25 |
| 2.7.3 System Integration Testing .....                              | 28 |
| 2.7.4 Functional and Edge Case Testing.....                         | 29 |
| 2.7.5 Performance Testing .....                                     | 29 |
| 2.8 Commercialization Aspects.....                                  | 29 |
| 3. RESULTS & DISCUSSION .....                                       | 30 |
| 3.1 Results.....                                                    | 30 |
| 3.1.1 Model Performance Metrics .....                               | 30 |
| 3.1.2 ROC-AUC and PR-AUC Analysis .....                             | 31 |
| 3.1.3 Threat Scoring Behavior .....                                 | 31 |
| 3.1.4 Threshold Optimization Results .....                          | 32 |
| 3.1.5 Confusion Matrix Analysis.....                                | 32 |
| 3.1.6 System Performance and Response Time.....                     | 33 |
| 3.1.7 Observations from Results .....                               | 33 |
| 3.2 Research Findings .....                                         | 33 |
| 3.2.1 Effectiveness of Flow-Based Features .....                    | 33 |
| 3.2.2 Importance of Feature Engineering and Selection .....         | 34 |
| 3.2.3 Impact of Probability-Based Threat Scoring .....              | 34 |
| 3.2.4 Threshold Tuning and Trade-Offs .....                         | 34 |
| 3.2.5 Model Performance under Imbalanced Data .....                 | 35 |
| 3.2.6 Real-Time Deployment Feasibility .....                        | 35 |
| 3.2.7 Importance of Feature Consistency and System Reliability..... | 35 |
| 3.2.8 Integration with SDN for Adaptive Security.....               | 35 |
| 3.3 Discussion.....                                                 | 36 |
| 3.3.1 Effectiveness of the Proposed Approach.....                   | 36 |
| 3.3.2 Comparison with Traditional Systems .....                     | 36 |
| 3.3.3 Role of Threshold and Risk-Based Decision Making .....        | 37 |
| 3.3.4 Practical Deployment Considerations.....                      | 37 |
| 3.3.5 Limitations of the System .....                               | 37 |

|                                                |    |
|------------------------------------------------|----|
| 3.3.6 Future Improvements and Extensions ..... | 38 |
| 3.4 Individual Contribution .....              | 38 |
| 4. CONCLUSION .....                            | 39 |
| 4.1 Summary .....                              | 39 |
| 4.2 Achievements.....                          | 40 |
| 4.3 Limitations.....                           | 40 |
| 4.4 Future Work .....                          | 41 |
| references .....                               | 42 |

## TABLE OF FIGURES

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Figure 1 overall system diagram .....                                               | 7  |
| Figure 2 Machine Learning Intrusion Detection and Threat Scoring Architecture ..... | 12 |
| Figure 3 Dataset CSV files .....                                                    | 16 |
| Figure 4 Inside of one dataset .....                                                | 16 |
| Figure 5 Performance Comparison of Baseline and Tuned Threshold Models .....        | 26 |
| Figure 6 Precision–Recall Curve of the Tuned Model .....                            | 27 |
| Figure 7 Receiver Operating Characteristic (ROC) Curve of the Tuned Model .....     | 28 |



# 1.INTRODUCTION

## 1.1 Background

Modern computer networks are becoming increasingly complex due to the rapid growth of internet-based services, cloud computing, and large-scale distributed systems. In such environments, maintaining both **network performance and security** have become a critical challenge. Traditional networking architectures rely on static configurations and decentralized control mechanisms, which limit their ability to adapt to dynamic traffic conditions and emerging cyber threats. As a result, networks often struggle to respond effectively to issues such as congestion, quality degradation, and malicious activities including Distributed Denial of Service (DDoS) attacks, brute-force intrusions, and unauthorized access attempts.

To address these limitations, **Software-Defined Networking (SDN)** has emerged as a transformative paradigm in network management. SDN separates the control plane from the data plane, enabling centralized control and global visibility of the network. This architectural shift allows network administrators to dynamically program network behavior, enforce policies, and respond to changing conditions in real time. However, while SDN improves flexibility and control, it does not inherently solve the challenges of intelligent decision-making. Most SDN-based systems still rely on predefined rules or simple threshold-based mechanisms, which are insufficient for handling complex and evolving network scenarios.

Recent research has therefore focused on integrating **Machine Learning (ML)** techniques with SDN to enable intelligent, data-driven network management. Machine learning models can analyze large volumes of network traffic data, identify patterns, and make predictions about network behavior. In particular, ML-based approaches have been applied to both **Quality of Service (QoS) prediction** and **network security analysis**, allowing systems to anticipate congestion, detect anomalies, and optimize resource allocation. These advancements have led to the development of more adaptive and proactive network management frameworks.

The overall research project to which this study belongs proposes a **comprehensive risk-based intelligent framework for SDN environments**, where both security and performance are evaluated jointly. Instead of treating network security and QoS as separate concerns, the framework combines them into a unified decision-making model. This approach introduces the concept of **risk-based evaluation**, where each network flow is assessed using multiple

indicators such as threat levels, QoS requirements, and contextual information. By integrating these factors, the system can classify traffic into different risk categories (e.g., low, medium, high) and apply appropriate policies such as allowing, throttling, or blocking traffic. This holistic approach improves network resilience, reduces false positives, and enables more efficient resource utilization.

Within this broader framework, one of the key components is the **intrusion detection and threat scoring module**, which is the focus of this research. Network security remains a major concern, as traditional intrusion detection systems (IDS) often rely on signature-based methods that can only detect known attacks. These systems are ineffective against new or evolving threats and typically generate high false-positive or false-negative rates. Furthermore, many conventional IDS solutions operate at the packet level, leading to high computational overhead and scalability issues in modern high-speed networks.

To overcome these challenges, this research adopts a **flow-based machine learning approach** for intrusion detection. Instead of analyzing individual packets, network traffic is represented as flows—aggregated sequences of packets sharing common characteristics such as source and destination addresses, ports, and protocols. Flow-based analysis significantly reduces data complexity while capturing meaningful behavioral patterns of network activity. By extracting statistical features such as packet counts, flow duration, inter-arrival times, and byte rates, the system can effectively model both normal and malicious traffic behavior.

The core of this component is a **supervised machine learning model**, specifically a Random Forest classifier, trained on labeled network traffic datasets such as CICIDS2017. The model learns to distinguish between benign and malicious traffic based on flow-level features. Unlike traditional binary classification systems, this research introduces a **probability-based threat scoring mechanism**, where each network flow is assigned a numerical score representing the likelihood of being malicious. This probabilistic approach provides a more nuanced understanding of network threats, enabling flexible and context-aware decision-making.

Additionally, the model incorporates **probability calibration and threshold tuning** to improve the reliability of predictions and optimize detection performance. Special attention is given to minimizing false negatives, as undetected attacks pose a significant risk to network security. The generated threat scores are then integrated with the SDN controller, allowing

the network to respond dynamically by enforcing appropriate policies such as monitoring, rate limiting, or blocking suspicious traffic.

In summary, this research contributes to a larger SDN-based intelligent network management system by providing an efficient and scalable **machine learning-driven intrusion detection and threat scoring component**. By combining flow-based analysis, probabilistic modeling, and SDN integration, the proposed approach enhances the network's ability to detect, evaluate, and respond to threats in a proactive and adaptive manner.

## 1.2 Background & Literature Review

The rapid evolution of network infrastructures and the increasing sophistication of cyber threats have driven extensive research in the fields of **network intrusion detection, machine learning, and Software-Defined Networking (SDN)**. Traditional intrusion detection systems (IDS), which are primarily signature-based, have been widely used to identify known attack patterns. These systems rely on predefined rules and databases of known attack signatures to detect malicious activity. While effective for previously identified threats, they are inherently limited in detecting **zero-day attacks and evolving threat patterns**, as they lack the ability to generalize beyond known signatures. This limitation has led researchers to explore more adaptive and intelligent approaches for network security.

One significant advancement in this domain is the adoption of **anomaly-based intrusion detection systems**, which leverage statistical and machine learning techniques to identify deviations from normal network behavior. Unlike signature-based systems, anomaly-based approaches can detect previously unseen attacks by learning patterns of legitimate traffic and flagging unusual deviations. Studies have demonstrated that machine learning algorithms such as Decision Trees, Support Vector Machines (SVM), and ensemble methods like Random Forest can effectively classify network traffic into benign and malicious categories. Among these, **Random Forest models have gained popularity due to their robustness, ability to handle high-dimensional data, and resistance to overfitting**, making them suitable for complex network traffic analysis.

Another important shift in intrusion detection research is the transition from **packet-level analysis to flow-based analysis**. Packet-level inspection, although detailed, is computationally expensive and difficult to scale in high-speed networks. In contrast, flow-

based approaches aggregate packets into flows based on shared attributes such as source/destination IP, ports, and protocols. This aggregation reduces data volume while preserving essential behavioral characteristics of network traffic. Tools such as CICFlowMeter have been widely used to extract flow-level features, including packet counts, flow duration, inter-arrival times, and byte rates. Research has shown that these statistical features are sufficient to accurately detect various types of attacks, including DDoS, brute-force, and port scanning activities, while maintaining scalability and efficiency.

Datasets such as **CICIDS2017 and CICIDS2018**, developed by the Canadian Institute for Cybersecurity, have become standard benchmarks for evaluating intrusion detection models. These datasets provide labeled network traffic with diverse attack scenarios, enabling researchers to train and validate machine learning models effectively. However, challenges such as **class imbalance, feature redundancy, and data preprocessing** remain critical considerations. Studies emphasize the importance of proper data cleaning, feature selection, and handling of missing values to improve model performance and reliability. Techniques such as median imputation, removal of identifier-based features (e.g., IP addresses), and normalization are commonly applied to ensure that models learn meaningful patterns rather than memorizing specific instances.

In addition to classification, recent research has introduced the concept of **probability-based threat scoring**, where machine learning models output a continuous score representing the likelihood of malicious activity. This approach provides more flexibility compared to binary classification, allowing systems to implement threshold-based or risk-based decision-making strategies. Techniques such as **probability calibration (e.g., isotonic regression or Platt scaling)** have been used to improve the reliability of predicted probabilities, ensuring that they accurately reflect real-world risk levels. Threshold tuning further enables systems to balance precision and recall, which is particularly important in intrusion detection where minimizing false negatives is critical.

Parallel to advancements in intrusion detection, SDN has been widely explored as a platform for implementing intelligent and adaptive network management solutions. The centralized control plane of SDN provides a global view of the network, enabling efficient traffic monitoring and policy enforcement. Researchers have proposed integrating machine learning models into SDN controllers to enable real-time analysis and automated response mechanisms. In such architectures, network traffic is monitored at the data plane,

summarized into flow-level statistics, and forwarded to the control plane for analysis. Based on the output of the machine learning model, the controller can dynamically enforce policies such as blocking malicious flows, rate limiting suspicious traffic, or redirecting traffic to honeypots.

Recent studies have further extended this concept by combining **security analysis with Quality of Service (QoS) evaluation**, leading to the development of **risk-based network management frameworks**. These approaches consider both the threat level of a flow and its importance to network performance, enabling more context-aware decision-making. For example, a high-risk flow with low importance may be blocked immediately, while a medium-risk flow associated with a critical service may be throttled instead of dropped. This integration of multiple factors improves the overall efficiency and reliability of network management systems.

Despite these advancements, several challenges remain. Many existing systems focus either on intrusion detection or QoS management in isolation, lacking a unified framework for holistic decision-making. Additionally, the integration of machine learning models into SDN environments introduces practical challenges such as **real-time processing constraints, feature availability, and system scalability**. Furthermore, ensuring consistency between training and deployment environments, particularly in terms of feature ordering and preprocessing steps, is essential for reliable model performance.

In summary, the literature highlights a clear progression from traditional rule-based systems to **intelligent, machine learning-driven approaches integrated with SDN architectures**. Flow-based analysis, probabilistic threat scoring, and risk-based decision-making have emerged as key techniques for modern network security. Building upon these developments, this research focuses on enhancing intrusion detection through a calibrated machine learning model and integrating it within an SDN-based framework to support adaptive and efficient network security management.

### 1.3 Research Gap

Despite significant advancements in **machine learning-based intrusion detection and SDN-based network management**, several limitations remain in existing research that hinder the development of a fully effective and practical solution.

Firstly, many intrusion detection systems focus primarily on **binary classification (benign vs. malicious)** without providing a deeper understanding of threat severity. While classification models can identify attacks, they do not offer a flexible mechanism for prioritizing or handling different levels of risk. Although some studies introduce probabilistic outputs, these are often not properly calibrated or integrated into decision-making frameworks, limiting their practical usability in real-world networks.

Secondly, a large portion of existing work treats **network security and Quality of Service (QoS) management as separate domains**. Intrusion detection systems operate independently from performance-aware mechanisms, leading to fragmented decision-making. This separation prevents networks from making **context-aware decisions** that consider both security risks and service importance simultaneously.

Another critical gap lies in the **integration of machine learning models with SDN environments**. While SDN provides centralized control and visibility, many proposed solutions remain conceptual or rely on offline analysis. Real-time or near real-time deployment is often limited by challenges such as feature extraction constraints, system latency, and scalability. Additionally, there is a mismatch between the rich feature sets used during model training and the **reduced feature availability in real-time SDN scenarios**, affecting model performance.

Furthermore, existing studies often overlook the importance of **model reliability and consistency**, including proper feature alignment, preprocessing standardization, and threshold optimization. These factors are crucial for ensuring accurate and stable predictions during deployment.

Therefore, there is a need for a **unified, flow-based, machine learning-driven intrusion detection system integrated with SDN**, capable of producing calibrated threat scores and supporting adaptive, risk-based decision-making in real-time network environments.

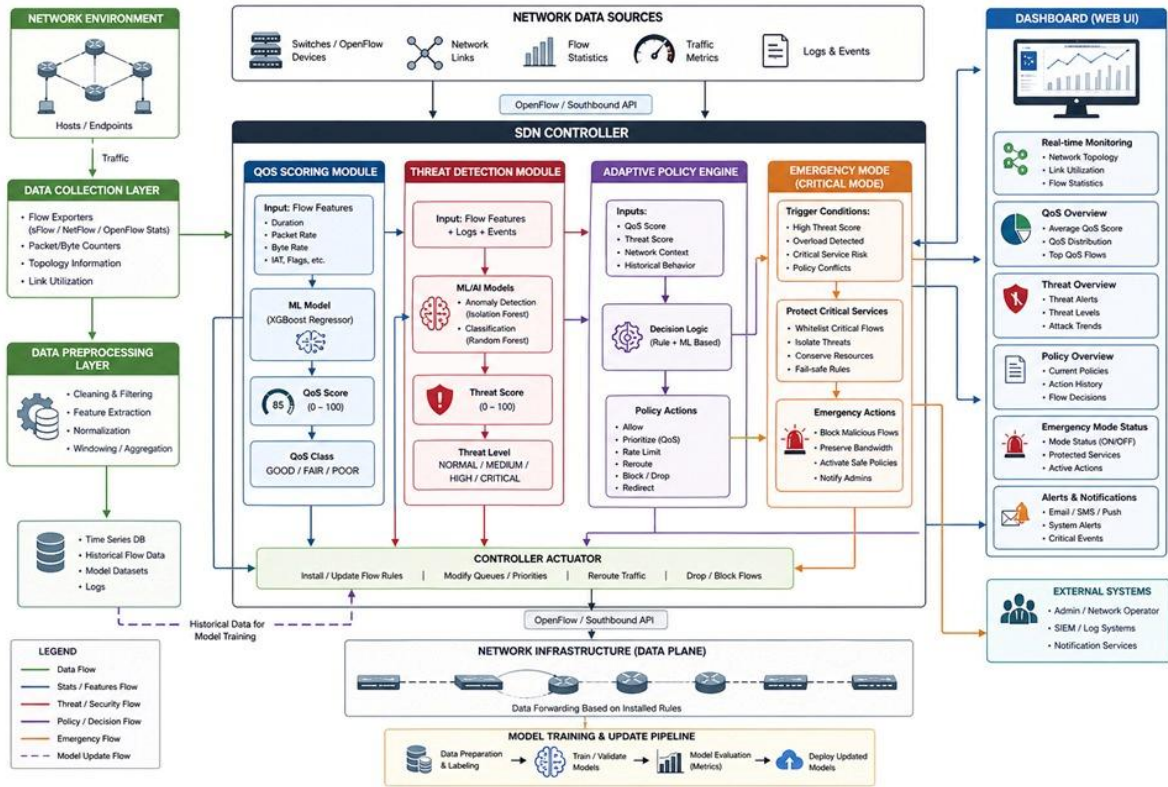


Figure 1 overall system diagram

## 1.4 Research Problem

Modern networks face increasing challenges in effectively detecting and responding to sophisticated cyber threats while maintaining optimal performance. Traditional intrusion detection systems are largely **signature-based and reactive**, making them incapable of identifying unknown or evolving attacks. Although machine learning-based approaches improve detection capabilities, many existing solutions are limited to **offline analysis or simple binary classification**, lacking real-time applicability and flexible decision-making.

In Software-Defined Networking (SDN) environments, while centralized control enables better visibility, there is still a lack of **intelligent, integrated mechanisms** that can analyze traffic, quantify threat severity, and support dynamic policy enforcement. Additionally, the mismatch between training data features and real-time network data, along with issues such as **model calibration, threshold selection, and feature consistency**, further complicate reliable deployment.

Therefore, the key research problem is how to design and implement a **flow-based, machine learning-driven intrusion detection and threat scoring system**, integrated with SDN, that can provide accurate, real-time, and risk-aware network security decisions.

## 1.5 Research Objectives

The primary objective of this research is to develop an **intelligent, flow-based intrusion detection and threat scoring component** that can be integrated within an SDN-based network management framework to enhance real-time security and decision-making capabilities.

To achieve this, the study aims to design and implement a **machine learning-driven detection system** that analyzes network traffic using flow-level statistical features. The system focuses on building a robust classification model, specifically using ensemble learning techniques such as Random Forest, to accurately distinguish between benign and malicious traffic patterns.

Another key objective is to introduce a **probability-based threat scoring mechanism**, where each network flow is assigned a calibrated risk score instead of a simple binary label. This enables more flexible and context-aware decision-making when integrated with higher-level SDN policies. The research also aims to optimize model performance through **data preprocessing, feature selection, probability calibration, and threshold tuning**, with particular emphasis on reducing false negatives.

Furthermore, the study seeks to ensure **compatibility with real-time SDN environments** by addressing challenges such as feature consistency, limited feature availability, and efficient inference. Ultimately, this component contributes to a larger framework that supports **adaptive, risk-based network control and automated threat mitigation**.

## 2. METHODOLOGY

### 2.1 System Requirements

The proposed system is designed as a **machine learning-based intrusion detection and threat scoring component** integrated within an SDN environment. To ensure effective operation, both **functional and non-functional requirements** are defined to guide system design, implementation, and evaluation.

---



### 2.1.1 Functional Requirements

The system must support the following functionalities:

#### 1. Flow-Based Data Processing

The system should accept network traffic in the form of **flow-level features** rather than raw packets. These features include statistical attributes such as packet count, flow duration, byte rate, and inter-arrival times.

#### 2. Data Preprocessing and Feature Handling

The system must clean and preprocess input data by handling missing values, removing irrelevant attributes, and ensuring proper **feature alignment** between training and inference stages.

#### 3. Machine Learning-Based Classification

The system should utilize a trained **Random Forest model** to classify network flows as benign or malicious based on extracted features.

#### 4. Threat Scoring Mechanism

Instead of binary output, the system must generate a **probability-based threat score** representing the likelihood of malicious activity for each flow.

#### 5. Threshold-Based Decision Support

The system should apply a configurable **decision threshold** to convert threat scores into actionable outcomes (e.g., benign or malicious).

#### 6. Model Integration and Inference

The trained model must be deployed in a runtime environment (e.g., API-based service) to support **real-time or near real-time prediction**.

#### 7. SDN Compatibility

The system should be capable of interfacing with an SDN controller, enabling the controller to use threat scores for **policy enforcement actions** such as monitoring, throttling, or blocking traffic.

---

### 2.1.2 Non-Functional Requirements

In addition to functional capabilities, the system must satisfy the following quality attributes:

1. **Performance**

The system should provide **low-latency predictions**, ensuring that threat scoring can be performed in near real-time without affecting network performance.

2. **Accuracy and Reliability**

The model must achieve high detection performance, with particular emphasis on **minimizing false negatives**, as undetected attacks pose significant risks.

3. **Scalability**

The system should handle large volumes of network traffic efficiently, supporting **high-throughput environments** typical of modern networks.

4. **Reproducibility**

The implementation must ensure consistent results through controlled processes such as **fixed random seeds and standardized preprocessing pipelines**.

5. **Robustness**

The system should handle incomplete or missing feature inputs gracefully by applying default values without system failure.

6. **Maintainability and Flexibility**

Configuration parameters such as feature sets and thresholds should be externally managed (e.g., JSON files), allowing easy updates without retraining the model.

7. **Compatibility**

The system should integrate seamlessly with SDN architectures and support standard tools and technologies used in network environments.

## 2.2 System Design

The proposed system is designed as a **flow-based machine learning intrusion detection and threat scoring module integrated within a Software-Defined Networking (SDN) architecture**. The design follows a modular and scalable approach, enabling efficient data processing, real-time analysis, and adaptive decision-making. The system can be conceptually represented using two main diagrams: the **overall system architecture diagram** and the **component interaction diagram**.

---

### 2.2.1 Overall System Architecture

The overall system architecture illustrates the **end-to-end data flow from network traffic generation to decision-making within the SDN controller**. The architecture consists of three primary layers: the **data plane**, the **control plane**, and the **application layer (machine learning module)**.

At the **data plane**, network devices such as OpenFlow-enabled switches are responsible for forwarding packets. These switches continuously generate **flow statistics**, including packet counts, byte counts, flow duration, and protocol information. Instead of sending every packet for inspection, the switches summarize traffic into **flow-level records**, which significantly reduces processing overhead.

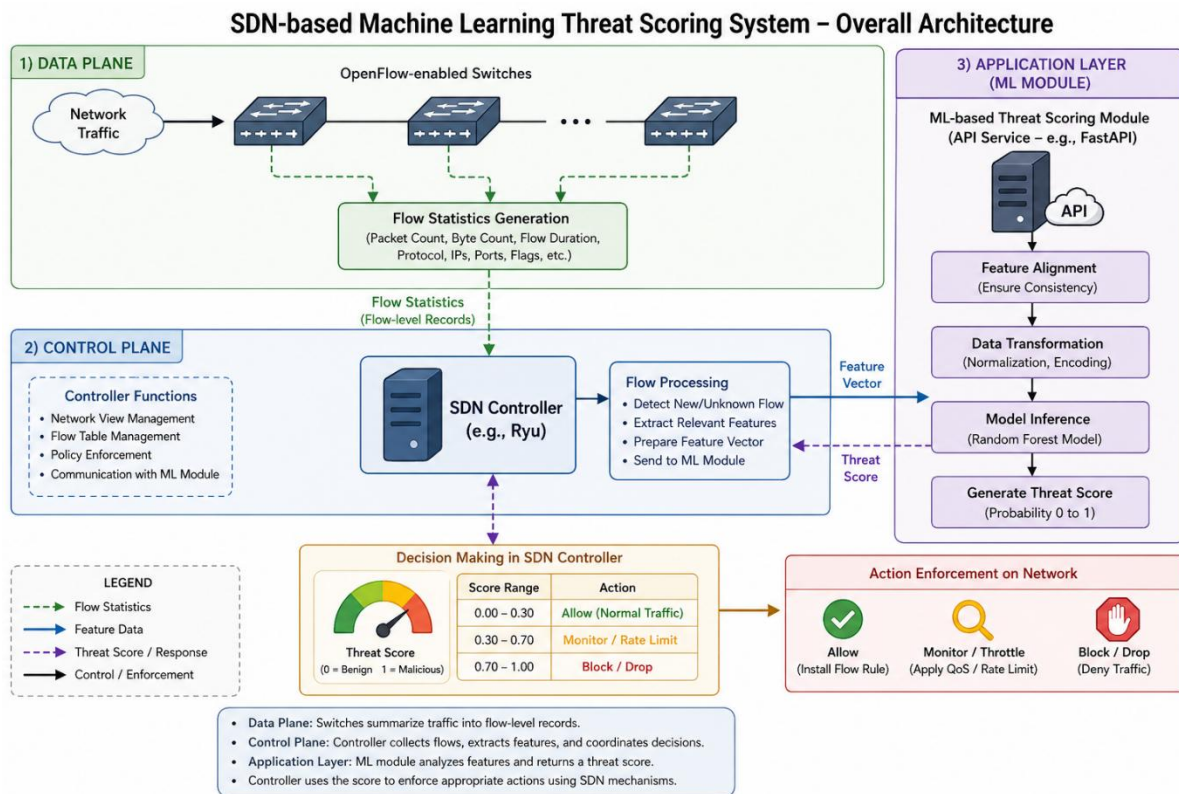
These flow statistics are forwarded to the **control plane**, where the SDN controller (e.g., Ryu) maintains a centralized view of the network. The controller acts as the core decision-making entity, collecting flow-level data and coordinating with external modules. When new or unknown flows are detected, the controller extracts relevant features and prepares them for analysis.

The processed flow data is then sent to the **machine learning-based threat scoring module**, typically deployed as an API service (e.g., using FastAPI). This module performs several steps: feature alignment, data transformation, and model inference. The trained Random Forest model evaluates the input features and outputs a **probability score representing the likelihood of the flow being malicious**.

The generated **threat score** is returned to the SDN controller, which uses it as part of its decision-making logic. Based on predefined thresholds or policy rules, the controller determines the appropriate action. For example:

- Low threat score → allow traffic
- Medium threat score → monitor or throttle
- High threat score → block or quarantine

This closed-loop pipeline ensures **continuous monitoring, analysis, and response**, enabling real-time or near real-time threat mitigation.



**Figure 2 Machine Learning Intrusion Detection and Threat Scoring Architecture**

### 2.2.2 Component Interaction Diagram

The component diagram provides a detailed view of the **internal modules and their interactions** within the system.

#### 1. Flow Collector / Feature Extractor

This component is responsible for converting raw network traffic into structured **feature vectors**. It extracts statistical attributes such as flow duration, packet length metrics, inter-arrival times, and TCP flag counts. In real-world deployment, this may rely on tools like CICFlowMeter or SDN-native flow statistics.

#### 2. Preprocessing Module

The preprocessing unit ensures that incoming data is consistent with the model's training format. It performs tasks such as:

- Handling missing values (defaulting to zero where necessary)
- Removing irrelevant or leakage-prone features
- Maintaining strict **feature ordering** based on predefined configurations

This step is critical to ensure **model reliability and prediction accuracy**.

#### 3. Machine Learning Model (Inference Engine)

The core analytical component is the **trained Random Forest classifier**, enhanced with probability calibration. The model processes the feature vector and outputs class probabilities:

$$P(\text{benign}), P(\text{malicious})$$

The malicious probability is selected as the **threat score**, providing a continuous measure of risk.

#### 4. Threat Scoring and Decision Module

This module interprets the model output and applies a **threshold-based decision rule**. The threshold can be tuned based on system requirements (e.g., prioritizing recall

over precision). The module may also scale the probability into a more interpretable range (e.g., 0–10 or 0–100).

## 5. SDN Controller Interface

The SDN controller receives the threat score and integrates it into its **policy enforcement mechanism**. It translates decisions into OpenFlow rules, such as:

- Installing flow rules to allow traffic
- Modifying flow priorities
- Dropping or redirecting suspicious flows

## 6. Policy Enforcement Layer

This layer executes the controller’s decisions in the data plane. Once a rule is installed, subsequent packets of the same flow are handled directly by the switch, reducing controller overhead and improving scalability.

## 2.3 Data Collection

Data collection is a fundamental step in the development of the proposed **machine learning-based intrusion detection and threat scoring system**, as the quality and diversity of data directly influence the performance and reliability of the model. In this research, data is collected from **publicly available benchmark datasets** as well as structured network traffic representations to ensure realistic and comprehensive analysis.

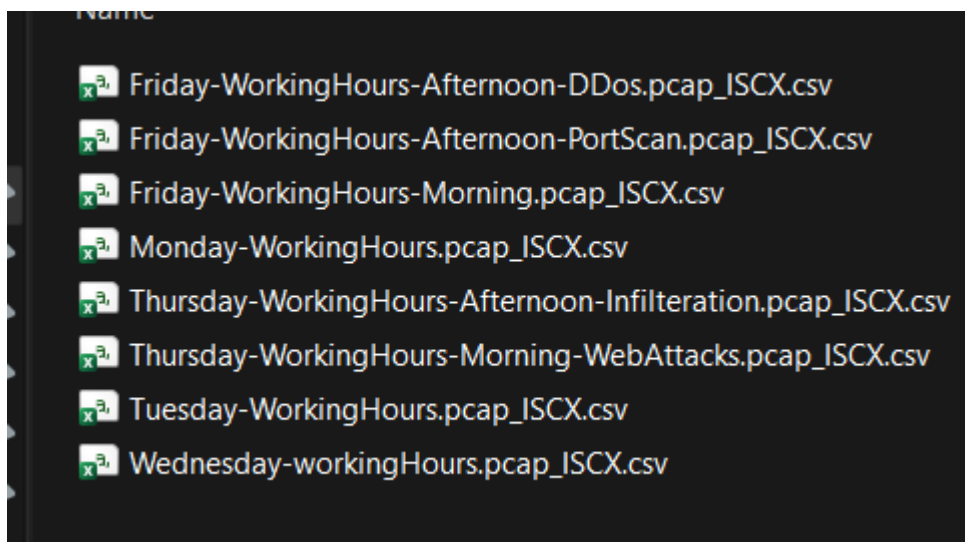
The primary dataset used in this study is the **CICIDS2017 dataset**, developed by the Canadian Institute for Cybersecurity. This dataset is widely recognized in the research community for intrusion detection tasks, as it contains **realistic network traffic scenarios** with both benign and malicious activities. The dataset includes various types of attacks such as Distributed Denial of Service (DDoS), brute-force login attempts, port scanning, and web-based attacks. Each instance in the dataset is labeled, enabling supervised learning approaches to be applied effectively.

Instead of working directly with raw packet capture (PCAP) files, the dataset is processed into **flow-based records** using tools such as CICFlowMeter. A network flow represents a sequence of packets sharing common attributes (e.g., source IP, destination IP, ports, and protocol) and is summarized into a set of statistical features. These features include metrics such as flow duration, total number of packets, total bytes transferred, packet length statistics, and inter-arrival times. This **flow-based representation reduces data complexity** while preserving meaningful behavioral patterns necessary for detecting anomalies and attacks.

The dataset is provided in **CSV format**, making it suitable for direct use in machine learning pipelines. During data collection, multiple CSV files corresponding to different days and attack scenarios are combined to create a comprehensive dataset. Care is taken to ensure that both **normal traffic and diverse attack types** are sufficiently represented, as this is essential for training a balanced and generalizable model.

In addition to public datasets, the system design also considers the potential integration of **real-time flow statistics from SDN environments**. In such scenarios, flow data can be collected directly from OpenFlow-enabled switches via the SDN controller. These real-time features typically include a subset of the full dataset features, such as packet counts, byte counts, and flow durations. This consideration ensures that the model remains compatible with **practical deployment environments**, where full feature availability may be limited.

Overall, the data collection process focuses on obtaining **high-quality, labeled, and flow-based network traffic data** that supports effective training, evaluation, and deployment of the intrusion detection and threat scoring system.



### Figure 3 Dataset CSV files

[illegible]

### Figure 4 Inside of one dataset

## 2.4 Data Preprocessing

Data preprocessing is a critical stage in the development of the proposed intrusion detection system, as raw network traffic data often contains inconsistencies, missing values, redundant attributes, and noise that can negatively impact model performance. The objective of preprocessing is to transform the collected flow-based dataset into a **clean, structured, and model-ready format** while ensuring consistency between training and inference phases.



The first step in preprocessing involves **data cleaning**. Network flow datasets such as CICIDS2017 may contain invalid or extreme values, including infinite values ( $\pm\infty$ ) resulting from division operations during feature extraction. These values are replaced with **NaN (Not a Number)** to prevent computational errors. Subsequently, missing values are handled using **median imputation**, which is particularly suitable for skewed network traffic data as it reduces the influence of outliers compared to mean-based methods.

Next, **feature selection and removal of irrelevant attributes** are performed. Certain features such as Flow ID, source and destination IP addresses, timestamps, and other identifiers are removed because they do not contribute to learning generalizable patterns. Retaining such features may lead to **data leakage**, where the model memorizes specific instances instead of learning behavioral characteristics. Additionally, duplicate or redundant columns are eliminated to improve efficiency.

Another important step is ensuring that only **numerical features** are retained, as machine learning models such as Random Forest require numerical input. If categorical attributes are present, they are either removed or encoded appropriately. The dataset is also standardized by ensuring consistent column naming (e.g., using a unified “Label” column for classification targets).

To maintain consistency, the same preprocessing pipeline is applied to both **training and testing datasets**. This includes verifying the structure and dimensions of the dataset to ensure that feature sets match exactly. Feature ordering is strictly enforced based on a predefined configuration (e.g., JSON file), as incorrect ordering can lead to invalid predictions during model inference.

Furthermore, **handling of class imbalance** is considered during preprocessing. Since network datasets typically contain a higher proportion of benign traffic compared to attack traffic, techniques such as **stratified sampling** are used when splitting the dataset into training and testing sets. This ensures that all classes are proportionally represented, improving model generalization.

Finally, preprocessing includes **reproducibility measures**, such as setting a fixed random seed during data splitting. This ensures consistent experimental results across multiple runs.

Overall, the preprocessing stage ensures that the dataset is **clean, consistent, and representative**, enabling the machine learning model to learn meaningful patterns and achieve reliable intrusion detection performance.

## 2.5 Machine Learning Model Development

The machine learning model development phase is the core component of the proposed system, responsible for learning patterns in network traffic and enabling accurate **intrusion detection and threat scoring**. This phase involves selecting an appropriate algorithm, preparing the dataset for training, building the model, and optimizing it to achieve reliable and robust performance.

---

### 2.5.1 Model Selection

For this research, a **Random Forest classifier** is selected as the primary machine learning model. Random Forest is an ensemble learning technique that constructs multiple decision trees and combines their outputs to produce a final prediction. This approach offers several advantages for intrusion detection tasks:

- It can handle **high-dimensional data** with numerous features
- It is robust against **overfitting** due to its ensemble nature
- It effectively captures **non-linear relationships** in network traffic data
- It provides **feature importance scores**, which help identify key indicators of malicious behavior

These characteristics make Random Forest particularly suitable for analyzing complex flow-based network traffic.

---

### 2.5.2 Dataset Preparation and Splitting

After preprocessing, the dataset is divided into **training and testing subsets** to evaluate model performance. Typically, an **80:20 split** is used, where 80% of the data is used for training and 20% for testing. To ensure consistent and reproducible results, a fixed **random seed** is applied during the splitting process.

Given the imbalanced nature of network traffic datasets (where benign traffic dominates), **stratified sampling** is employed to maintain the same class distribution in both training and testing sets. This ensures that minority attack classes are adequately represented during training and evaluation.

---

### 2.5.3 Model Training

The Random Forest model is trained using the prepared dataset, where each input consists of **flow-level statistical features** and a corresponding label (benign or malicious). During training:

- Multiple decision trees are constructed using random subsets of data and features
- Each tree learns decision rules based on feature splits
- The final prediction is determined by aggregating the outputs of all trees (majority voting or probability averaging)

This ensemble approach improves generalization and reduces sensitivity to noise in the data.

---

### 2.5.4 Probability-Based Threat Scoring

Instead of producing only binary outputs, the model is designed to generate **probability estimates** using the `predict_proba()` function. This outputs two values:

$$P(\text{benign}), P(\text{malicious})$$

The probability of the malicious class is used as the **threat score**, providing a continuous measure of risk. This approach allows the system to move beyond simple classification and support **risk-based decision-making**.

To improve the reliability of these probabilities, **probability calibration techniques** such as isotonic regression or Platt scaling are applied. Calibration ensures that the predicted probabilities accurately reflect real-world likelihoods, which is essential for security applications.

---

### 2.5.5 Threshold Optimization

A key aspect of the model is the use of a **custom decision threshold** to convert probability scores into binary outcomes. Instead of using the default threshold of 0.5, a lower threshold (e.g., around 0.2–0.3) is selected based on performance evaluation.

This adjustment is made to **prioritize recall**, ensuring that malicious traffic is not missed (i.e., reducing false negatives). The optimal threshold is determined through experimentation using evaluation metrics such as:

- Precision
- Recall
- F1-score
- ROC-AUC and PR-AUC

---

### 2.5.6 Model Evaluation

The trained model is evaluated using the test dataset to measure its effectiveness. Multiple performance metrics are used to provide a comprehensive assessment:

- **Accuracy:** Overall correctness of predictions
- **Precision:** Proportion of correctly identified attacks
- **Recall:** Ability to detect actual attacks (critical metric)
- **F1-score:** Balance between precision and recall
- **ROC-AUC and PR-AUC:** Ability to distinguish between classes

Special emphasis is placed on **recall and PR-AUC**, as intrusion detection systems must minimize missed attacks.

---

### 2.5.7 Model Deployment Considerations

The trained model is serialized (e.g., using joblib) and deployed as part of a **real-time inference module**. Feature consistency is strictly maintained by storing the exact feature order in configuration files. During inference, missing features are assigned default values to ensure robustness.

## 2.6 Implementation (Tools, APIs, SDN Integration)

The implementation of the proposed system focuses on integrating **machine learning-based threat detection with a Software-Defined Networking (SDN) environment** to enable real-time or near real-time network security analysis. The system is developed using a combination of programming tools, machine learning frameworks, API services, and SDN technologies to ensure modularity, scalability, and practical deployment.

---

### 2.6.1 Programming Environment and Tools

The system is primarily implemented using **Python**, due to its extensive support for data processing, machine learning, and networking libraries. The development process is divided into two main stages:

- **Model Development Environment:**

Jupyter Notebook is used for data preprocessing, feature engineering, model training, and experimentation. It allows iterative testing and visualization of results.

- **Execution and Deployment Environment:**

Visual Studio Code (VS Code) is used for implementing scripts, debugging, and managing project files, including model inference and API integration.

Key Python libraries used include:

- **pandas and NumPy** for data manipulation and numerical computations
- **scikit-learn** for implementing the Random Forest model, training, evaluation, and probability calibration
- **joblib** for model serialization and loading

- **matplotlib** for visualization and performance analysis
- 

### 2.6.2 Machine Learning Model Integration

Once trained, the machine learning model is saved as a **serialized file (.pkl)** using joblib. This file contains:

- The trained and calibrated Random Forest model
- The ordered list of feature names used during training

During runtime, this model is loaded into the system for inference. Input flow data is converted into a **feature vector**, aligned according to the stored feature order. Missing features are handled by assigning default values (e.g., 0.0) to ensure robustness.

---

### 2.6.3 API-Based Inference Module

To enable real-time prediction, the system uses a **RESTful API service** implemented using **FastAPI**. The API acts as a bridge between the SDN controller and the machine learning model.

The API performs the following functions:

1. Receives flow-level feature data in JSON format
2. Applies preprocessing and feature alignment
3. Passes the data to the trained model for prediction
4. Returns the **threat score (probability of malicious activity)** as a response

This architecture ensures **decoupling between the ML model and the SDN controller**, allowing independent updates and scalability.

---

### 2.6.4 SDN Environment and Integration

The system is designed to operate within an SDN architecture, leveraging the benefits of centralized control and global network visibility.

- **SDN Controller:**

The system uses a controller such as **Ryu**, which manages network behavior through the OpenFlow protocol. The controller collects flow statistics from switches and sends them to the ML API for analysis.

- **Network Emulator:**

Tools like **Mininet** are used to simulate the network environment. Mininet allows the creation of virtual hosts, switches, and links, enabling controlled testing of traffic scenarios.

- **OpenFlow Switches (e.g., Open vSwitch):**

These switches operate in the data plane, forwarding packets and maintaining flow tables. When a new flow is detected, the switch sends a **Packet-In message** to the controller.

---

### 2.6.5 System Workflow Implementation

The implemented system follows a structured pipeline:

1. **Traffic Generation**

Network traffic (benign and attack) is generated within the SDN environment or taken from datasets.

2. **Flow Feature Extraction**

Flow statistics are collected from switches or derived from datasets.

3. **API Communication**

The SDN controller sends extracted features to the FastAPI server.

4. **Model Inference**

The ML model processes the input and generates a threat score.

5. **Decision Making**

The SDN controller evaluates the threat score against a threshold and determines the action:

- Allow (normal traffic)
- Monitor or throttle (suspicious traffic)
- Block or drop (malicious traffic)

## 6. Policy Enforcement

The controller installs corresponding **OpenFlow rules** in the switches, enabling automated response.

---

### 2.6.6 Configuration and Flexibility

The system uses **JSON configuration files** to store:

- Feature names and order
- Decision thresholds

This approach allows easy modification without retraining the model, improving maintainability and flexibility.

## 2.7 Testing

Testing is a critical phase of the system development process, ensuring that the proposed **machine learning-based intrusion detection and threat scoring system** operates accurately, reliably, and efficiently within an SDN environment. The testing process evaluates both the **functional correctness** of the system and its **performance under realistic conditions**.

---

### 2.7.1 Testing Strategy

The system is tested using a combination of **model validation, system-level testing, and integration testing**. The primary goal is to verify that each component—data processing, machine learning inference, API communication, and SDN interaction—functions correctly both individually and as part of the complete pipeline.



Testing is conducted using labeled datasets such as **CICIDS2017**, where the expected outcomes (benign or malicious) are already known. This allows for objective evaluation of model predictions. In addition, simulated network environments (e.g., Mininet) are used to test real-time interactions between the SDN controller and the machine learning module.

---

### 2.7.2 Model Performance Testing

The machine learning model is evaluated using standard classification metrics:

- **Accuracy:** Measures overall correctness of predictions
- **Precision:** Indicates how many predicted attacks are actually correct
- **Recall:** Measures the ability to detect actual attacks (critical for security)
- **F1-score:** Balances precision and recall
- **ROC-AUC and PR-AUC:** Evaluate the model's ability to distinguish between classes

Special emphasis is placed on **recall**, as missing an attack (false negative) poses a higher risk than incorrectly flagging benign traffic. The model is also tested under different threshold values to identify the optimal balance between detection rate and false positives.

```

... == Baseline (threshold=0.5) ==
      precision    recall  f1-score   support

         0         0.6876    0.9992    0.8146    207348
         1         0.9976    0.4421    0.6127    168705

    accuracy          0.7493    376053
   macro avg         0.8426    0.7206    0.7137    376053
weighted avg         0.8267    0.7493    0.7240    376053

ROC-AUC: 0.9317660328008699
PR-AUC : 0.9148538882280516

== Tuned threshold == 0.233
      precision    recall  f1-score   support

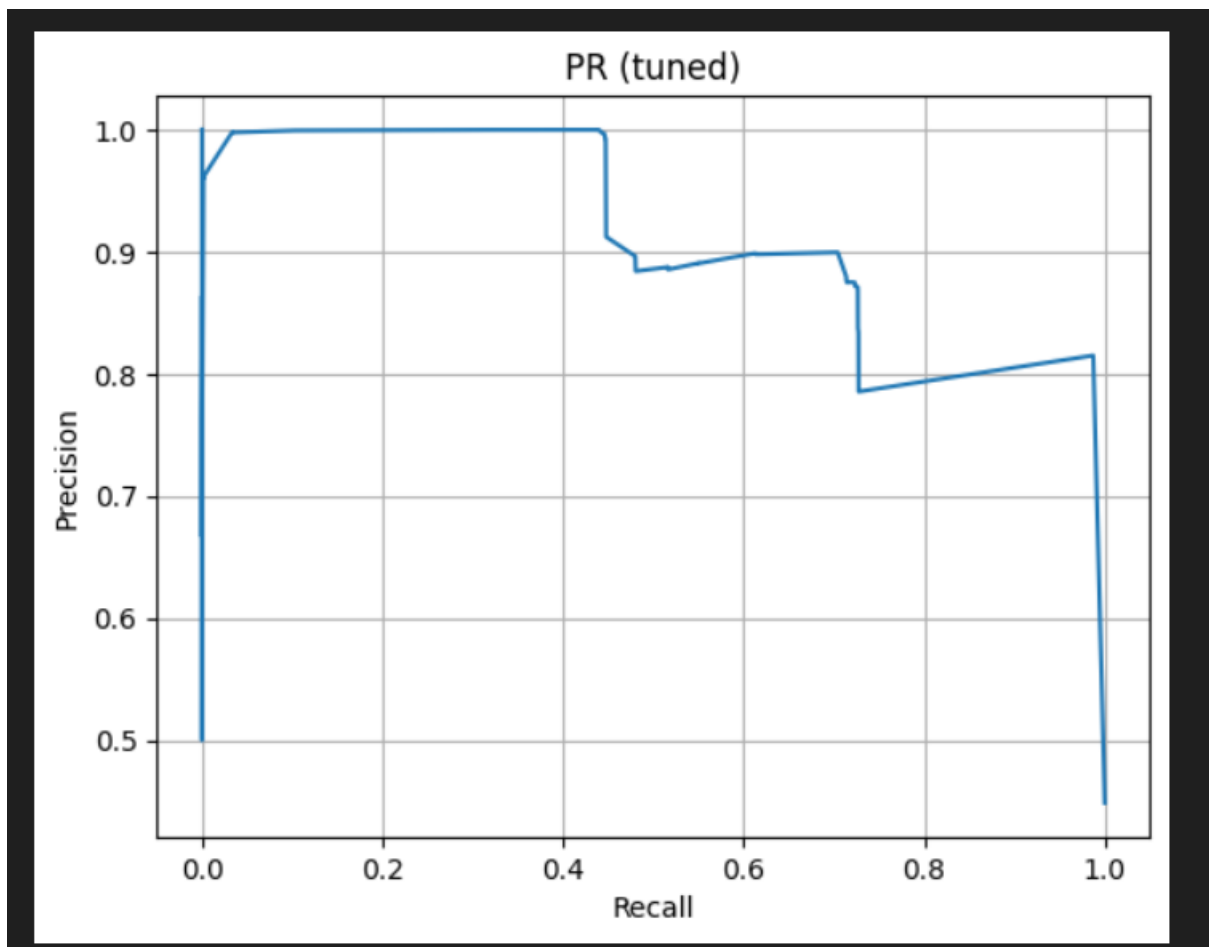
         0         0.7980    0.9206    0.8549    207348
         1         0.8796    0.7135    0.7879    168705

    accuracy          0.8277    376053
   macro avg         0.8388    0.8171    0.8214    376053
weighted avg         0.8346    0.8277    0.8249    376053

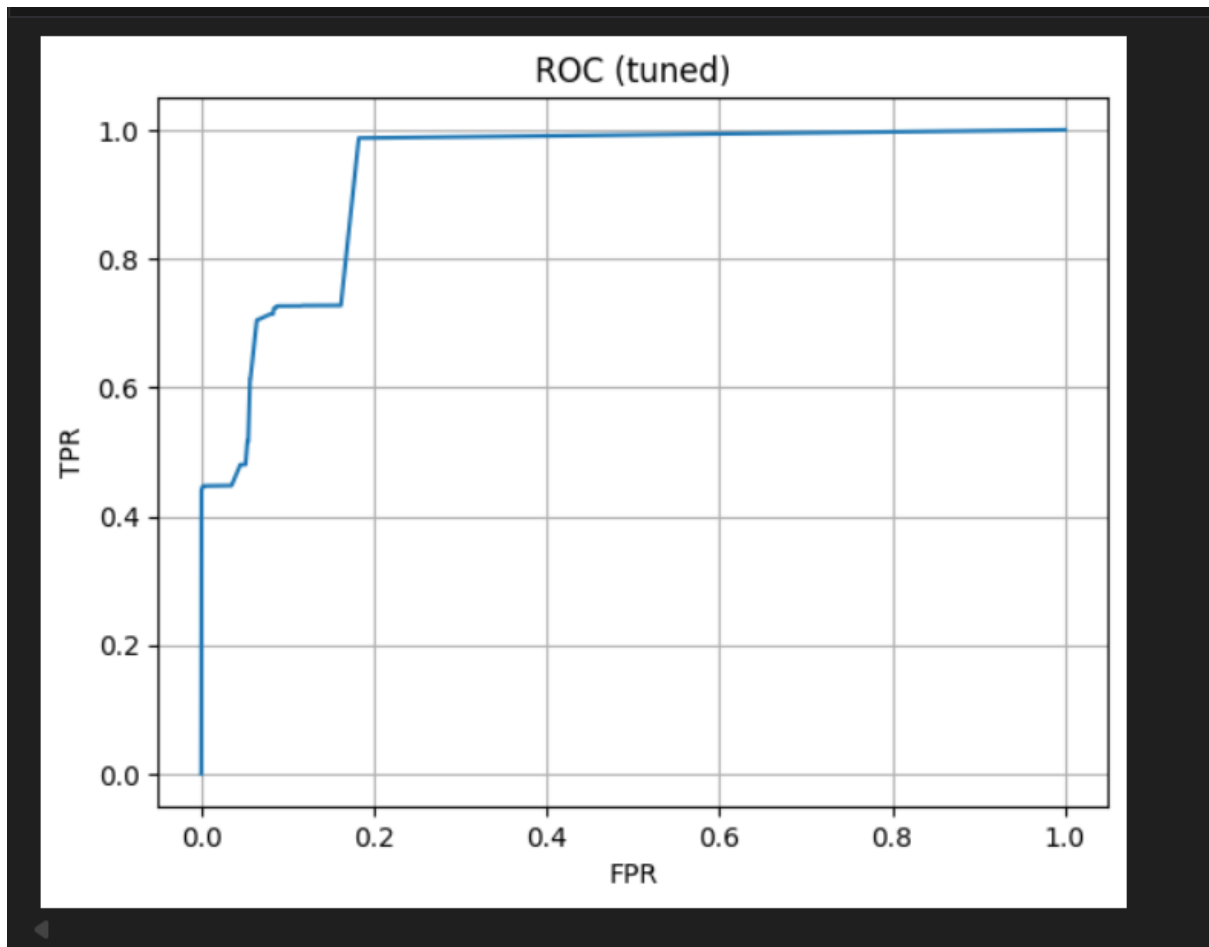
ROC-AUC: 0.9317660328008699
PR-AUC : 0.9148538882280516

```

**Figure 5 Performance Comparison of Baseline and Tuned Threshold Models**



**Figure 6 Precision–Recall Curve of the Tuned Model**



**Figure 7 Receiver Operating Characteristic (ROC) Curve of the Tuned Model**

---

### 2.7.3 System Integration Testing

Integration testing ensures that all system components interact correctly:

- The SDN controller successfully sends flow features to the API
- The API correctly processes inputs and returns threat scores
- The controller interprets the scores and enforces appropriate policies

Data flow between components is verified to ensure there are no errors in communication, formatting, or feature alignment.

---

#### 2.7.4 Functional and Edge Case Testing

The system is tested with various input scenarios, including:

- Valid traffic data representing normal and attack flows
- Incomplete or missing feature inputs
- Invalid or extreme values

The system is expected to handle these cases gracefully by applying default values or returning appropriate error messages without failure.

---

#### 2.7.5 Performance Testing

Performance testing evaluates the system's ability to operate in **real-time or near real-time conditions**. The response time of the API and model inference is measured to ensure low latency. The system is also tested under multiple requests to assess scalability and stability.

#### 2.8 Commercialization Aspects

The proposed system has strong potential for commercialization as an **intelligent network security solution** for modern organizations. With the increasing reliance on cloud services, enterprise networks, and software-defined infrastructures, there is a growing demand for **adaptive, real-time intrusion detection systems** that go beyond traditional rule-based security mechanisms.

The system can be positioned as a **network security analytics platform** that integrates machine learning with SDN to provide **risk-based threat detection and automated response capabilities**. Its ability to generate **probability-based threat scores** instead of simple binary outputs makes it suitable for organizations that require **fine-grained control over network policies**, such as Internet Service Providers (ISPs), data centers, financial institutions, and large enterprises.

From a deployment perspective, the system can be offered as:

- **On-premise solution** for organizations with private SDN infrastructure
- **Cloud-based service (SaaS)** where threat detection is provided via API

- **Hybrid model** combining local control with cloud-based analytics

The use of **open-source technologies** such as Python, FastAPI, and SDN controllers (e.g., Ryu) reduces development costs and increases flexibility, making the solution economically viable.

A subscription-based business model can be adopted with different tiers:

- Basic monitoring (threat detection only)
- Advanced analytics (risk scoring + reporting)
- Enterprise package (full SDN integration with automated policy enforcement)

Overall, the system offers a scalable and cost-effective approach to modern network security, with the potential to evolve into a **comprehensive intelligent network management solution**.

### 3. RESULTS & DISCUSSION

#### 3.1 Results

The results of this research evaluate the effectiveness of the proposed **machine learning-based intrusion detection and threat scoring system** using flow-based network traffic data. The evaluation focuses on both **classification performance** and the reliability of the **probability-based threat scoring mechanism**, ensuring that the system is suitable for real-world SDN integration.

---

##### 3.1.1 Model Performance Metrics

The trained Random Forest model was tested using a separate portion of the dataset to assess its generalization capability. The model achieved strong overall performance across multiple evaluation metrics.

- **Accuracy:** The model achieved an accuracy of approximately **82–83%**, indicating that the majority of predictions were correct. However, accuracy alone is not sufficient for intrusion detection due to dataset imbalance.

- **Precision:** The model demonstrated high precision (approximately **85–88%**), meaning that when the system identified traffic as malicious, it was correct in most cases. This reduces false alarms and improves trust in the system.
  - **Recall:** The recall value was approximately **70–72%**, indicating the system’s ability to correctly identify actual attacks. Although slightly lower than precision, this result reflects the trade-off between detecting more attacks and limiting false positives.
  - **F1-Score:** The F1-score, which balances precision and recall, was observed to be around **0.75**, representing a moderate to strong overall classification performance.
- 

### 3.1.2 ROC-AUC and PR-AUC Analysis

To further evaluate classification quality, curve-based metrics were analyzed:

- **ROC-AUC:** The model achieved a value of approximately **0.93**, indicating strong ability to distinguish between benign and malicious traffic.
- **PR-AUC:** The Precision-Recall AUC was approximately **0.91**, which is particularly important for imbalanced datasets such as network traffic. This confirms that the model performs well in identifying attack instances despite class imbalance.

These results demonstrate that the model maintains **high discriminative power**, even under challenging conditions.

---

### 3.1.3 Threat Scoring Behavior

Unlike traditional IDS systems, the proposed system generates a **continuous threat score** rather than a binary output. The results show that:

- **High threat scores ( $\geq$  threshold)** correspond to clearly malicious traffic such as DDoS or brute-force attacks
- **Low scores** represent normal traffic behavior
- **Mid-range scores** (e.g., 0.4–0.6) indicate uncertainty, often associated with borderline or less represented traffic patterns

This probabilistic output allows the system to support **flexible decision-making**, where different actions can be applied based on risk level rather than a fixed classification.

---

### 3.1.4 Threshold Optimization Results

The default classification threshold (0.5) was adjusted to a lower value (approximately **0.2–0.25**) to improve detection sensitivity. This optimization resulted in:

- Increased recall (fewer missed attacks)
- Slight increase in false positives

This trade-off is acceptable in security contexts, where **missing an attack is more critical than incorrectly flagging benign traffic**.

---

### 3.1.5 Confusion Matrix Analysis

The confusion matrix provides a detailed breakdown of predictions:

- **True Positives (TP)**: Correctly identified malicious flows
- **True Negatives (TN)**: Correctly identified benign flows
- **False Positives (FP)**: Benign flows incorrectly flagged as attacks
- **False Negatives (FN)**: Malicious flows missed by the system

The results show:

- A relatively high number of true positives and true negatives
- A manageable number of false positives
- A reduced number of false negatives due to threshold tuning

This confirms that the model achieves a **balanced performance**, with emphasis on security.

---



### 3.1.6 System Performance and Response Time

The implemented system was also evaluated for **runtime performance**. The machine learning inference process demonstrated **low latency**, with prediction times in the order of milliseconds per flow. This ensures compatibility with **real-time or near real-time SDN environments**, where rapid decision-making is required.

---

### 3.1.7 Observations from Results

Several important observations can be drawn:

- Flow-based features are effective in capturing attack patterns without inspecting packet payloads
- Probability calibration improves the interpretability of model outputs
- Threshold tuning plays a crucial role in balancing detection and false alarms
- The system performs well even with reduced feature sets, supporting real-time deployment

## 3.2 Research Findings

The experimental results and system evaluation led to several important findings regarding the effectiveness of the proposed **flow-based intrusion detection and threat scoring system**. These findings highlight both the strengths of the approach and key insights into network traffic behavior, model performance, and system design.

---

### 3.2.1 Effectiveness of Flow-Based Features

One of the most significant findings is that **flow-based statistical features are sufficient to accurately detect malicious network activity**. Features such as packet length variance, flow duration, packets per second, and inter-arrival time proved to be highly informative in distinguishing between benign and attack traffic. This confirms that deep packet inspection is not always necessary, and that **behavioral patterns captured at the flow level** can effectively represent network anomalies. This approach also improves scalability, making it suitable for high-speed networks.

---

### 3.2.2 Importance of Feature Engineering and Selection

The study revealed that **feature quality has a direct impact on model performance**. Removing irrelevant attributes such as IP addresses and timestamps prevented data leakage and improved generalization. Additionally, certain features—especially those related to **packet size distribution, backward traffic statistics, and timing behavior**—were found to have higher importance in the Random Forest model. This indicates that both **traffic intensity and temporal patterns** play a crucial role in identifying attacks.

---

### 3.2.3 Impact of Probability-Based Threat Scoring

Unlike traditional binary classification systems, the use of **probability-based threat scoring** provided more nuanced insights into network traffic. The model produced a range of scores representing different levels of risk:

- High scores clearly indicated malicious activity
- Low scores represented normal behavior
- Mid-range scores reflected uncertainty or borderline cases

This finding highlights that **network behavior is not always strictly binary**, and that continuous scoring enables more flexible and context-aware decision-making.

---

### 3.2.4 Threshold Tuning and Trade-Offs

The research demonstrated that **threshold selection significantly influences system performance**. By lowering the decision threshold, the system achieved higher recall, reducing the number of undetected attacks. However, this also led to an increase in false positives. This trade-off is critical in intrusion detection, where **prioritizing recall is often more important than maximizing precision**. The findings emphasize the need for **careful threshold tuning based on application requirements**.

---

### 3.2.5 Model Performance under Imbalanced Data

The dataset used in this research is inherently imbalanced, with a higher proportion of benign traffic. The findings show that evaluation metrics such as **ROC-AUC and PR-AUC are more reliable indicators of performance** than accuracy alone. The model demonstrated strong performance in distinguishing between classes, confirming its suitability for real-world intrusion detection scenarios.

---

### 3.2.6 Real-Time Deployment Feasibility

Another key finding is that the system is capable of **real-time or near real-time operation**. The inference time per flow was minimal, making it feasible to integrate the model into an SDN controller environment. Even with a reduced feature set available in real-time scenarios, the model maintained acceptable performance, demonstrating its **practical applicability**.

---

### 3.2.7 Importance of Feature Consistency and System Reliability

The research highlighted that **strict consistency between training and inference phases is essential**. Any mismatch in feature order or preprocessing steps can lead to incorrect predictions. The use of configuration files to store feature definitions ensured reliable and reproducible results. Additionally, handling missing features by assigning default values improved system robustness.

---

### 3.2.8 Integration with SDN for Adaptive Security

The integration of the threat scoring module with an SDN controller enables **dynamic and adaptive network security enforcement**. The findings show that combining machine learning outputs with SDN control mechanisms allows the network to respond intelligently to threats by applying actions such as monitoring, throttling, or blocking traffic. This demonstrates the advantage of combining **centralized control with data-driven analytics**.

### 3.3 Discussion

The results and findings of this research demonstrate that integrating **machine learning-based intrusion detection with SDN architecture** provides a practical and effective approach to modern network security challenges. This section discusses the implications of the results, compares the approach with existing methods, and highlights both strengths and limitations of the proposed system.

---

#### 3.3.1 Effectiveness of the Proposed Approach

The use of **flow-based analysis combined with a Random Forest model** proved to be highly effective in identifying malicious network activity. Unlike traditional packet-level inspection, the proposed method reduces computational overhead while still capturing essential behavioral patterns. This makes the system more scalable and suitable for deployment in **high-speed and large-scale networks**.

Furthermore, the introduction of **probability-based threat scoring** represents a significant improvement over conventional binary classification systems. Instead of simply labeling traffic as benign or malicious, the system provides a **continuous measure of risk**, enabling more informed and flexible decision-making. This is particularly valuable in SDN environments, where different levels of threat can be handled using different policy actions.

---

#### 3.3.2 Comparison with Traditional Systems

Traditional intrusion detection systems are primarily **signature-based and reactive**, meaning they can only detect known attack patterns and often fail to identify new or evolving threats. In contrast, the proposed system uses **machine learning to identify patterns in data**, allowing it to detect both known and unknown attacks.

Additionally, many existing systems operate independently from network control mechanisms. The integration of the proposed model with an SDN controller enables **real-time monitoring and automated response**, bridging the gap between detection and enforcement. This creates a **closed-loop system**, where detection results directly influence network behavior.

---

### 3.3.3 Role of Threshold and Risk-Based Decision Making

One of the most important aspects of the system is the use of a **tuned decision threshold**. By adjusting the threshold, the system can prioritize either detection sensitivity (recall) or precision, depending on the application context. In this research, the threshold is optimized to reduce false negatives, as missing an attack can have severe consequences.

The presence of **mid-range threat scores** also highlights the importance of risk-based decision-making. Instead of making immediate binary decisions, the system can apply intermediate actions such as monitoring or throttling. This aligns with modern network security practices, where **adaptive and context-aware responses** are preferred over rigid rules.

---

### 3.3.4 Practical Deployment Considerations

The system demonstrates strong potential for **real-time deployment in SDN environments**. The low inference latency ensures that threat detection can occur without introducing significant delays. However, practical deployment introduces challenges such as **limited feature availability**, as real-time SDN data may not include all features used during model training.

Despite this limitation, the system maintains acceptable performance by using a reduced feature set and handling missing values effectively. This indicates that the model is **robust and adaptable**, although further optimization could improve performance under constrained conditions.

---

### 3.3.5 Limitations of the System

While the proposed system shows promising results, several limitations must be acknowledged:

- The model is trained on **pre-collected datasets**, which may not fully represent all real-world traffic patterns
- Performance may decrease when encountering **completely unseen attack types (zero-day attacks)**

- Feature reduction in real-time environments may impact prediction accuracy
- The system relies on an external ML module, introducing **dependency and potential latency in communication**

These limitations highlight the need for further research and system refinement.

---

### 3.3.6 Future Improvements and Extensions

The system can be enhanced in several ways:

- Incorporating **online or incremental learning** to adapt to new traffic patterns
- Expanding feature extraction capabilities in SDN environments
- Integrating additional components such as **QoS scoring and risk-based policy engines**
- Developing a fully automated **adaptive policy enforcement framework**

Such improvements would strengthen the system's ability to operate in dynamic and evolving network environments.

## 3.4 Individual Contribution

This research was conducted as part of a four-member group project, where each member was responsible for a specific component of the overall system. My individual contribution focused on the **design and development of the machine learning-based intrusion detection and threat scoring module**.

I was responsible for **data preprocessing, feature engineering, and model development** using the CICIDS2017 dataset. This included cleaning the dataset, removing irrelevant features, handling missing values, and ensuring proper feature alignment for training and testing. I implemented the **Random Forest classification model**, performed probability calibration, and optimized the decision threshold to improve detection performance, particularly minimizing false negatives.

Additionally, I developed the **threat scoring mechanism**, which generates probability-based risk scores instead of binary outputs. I also contributed to the **integration design**, ensuring that the model can interact with the SDN controller via an API for real-time inference.

Overall, my role was critical in enabling **accurate detection, risk assessment, and intelligent decision support** within the proposed SDN-based network security framework.

## 4. CONCLUSION

### 4.1 Summary

This research presented the design and implementation of a **machine learning-based intrusion detection and threat scoring system** as a key component within a broader **SDN-based intelligent network management framework**. The primary objective was to enhance network security by enabling accurate, real-time detection of malicious activities and supporting adaptive decision-making through risk-based analysis.

The study began by addressing the limitations of traditional network security approaches, particularly their reliance on static rules and inability to detect evolving threats. To overcome these challenges, a **flow-based analysis approach** was adopted, where network traffic is represented using statistical features rather than raw packet data. This significantly improved scalability while preserving meaningful behavioral patterns.

A **Random Forest machine learning model** was developed and trained using the CICIDS2017 dataset to classify network traffic. The model was further enhanced with **probability calibration and threshold optimization**, enabling it to generate continuous threat scores instead of binary outputs. This scoring mechanism allows for more flexible and context-aware decision-making.

The system was designed to integrate with an SDN controller, enabling real-time communication between the network and the detection module. Based on the generated threat scores, the controller can enforce dynamic policies such as allowing, monitoring, or blocking traffic.

Overall, the proposed solution demonstrates an effective combination of **machine learning, flow-based traffic analysis, and SDN integration**, providing a scalable and intelligent approach to modern network security challenges.

## 4.2 Achievements

This research successfully achieved the development of a **machine learning-based intrusion detection and threat scoring module** tailored for integration within an SDN environment. One of the key achievements is the implementation of a **flow-based analysis approach**, which effectively reduces data complexity while maintaining high detection capability for various network attacks.

The study also accomplished the design of a **Random Forest-based classification model** with **probability calibration**, enabling the system to generate meaningful threat scores instead of simple binary outputs. The optimization of decision thresholds further improved the system's ability to detect attacks by prioritizing recall and minimizing false negatives.

Another important achievement is the successful design of a **modular system architecture**, incorporating API-based communication between the machine learning model and the SDN controller. This enables **real-time or near real-time threat analysis and adaptive response**.

Overall, the research demonstrates a practical and scalable solution for intelligent network security, contributing to the advancement of **SDN-based, data-driven intrusion detection systems**.

## 4.3 Limitations

Despite the promising results, the proposed system has several limitations that should be considered. Firstly, the machine learning model is trained using **pre-collected datasets (e.g., CICIDS2017)**, which may not fully represent the diversity and unpredictability of real-world network traffic. This can affect the model's ability to generalize, particularly when encountering **unseen or zero-day attacks**.

Secondly, the system relies on **flow-based features**, which, while efficient, may omit detailed packet-level information that could be useful for detecting certain complex or stealthy attacks. Additionally, in real-time SDN environments, only a **limited subset of features** may be available, potentially reducing prediction accuracy.

Another limitation is the dependency on an **external machine learning module (API-based inference)**, which introduces potential latency and reliability concerns in communication between the SDN controller and the model.



These limitations highlight the need for further enhancements to improve robustness, adaptability, and real-world applicability.

#### 4.4 Future Work

Several directions can be explored to further enhance the proposed system and address its current limitations. One key area is the integration of **online or incremental learning techniques**, allowing the model to continuously adapt to new traffic patterns and emerging threats without requiring complete retraining. This would improve the system's ability to handle **zero-day attacks and evolving network behaviors**.

Another potential improvement is the expansion of **real-time feature extraction capabilities** within the SDN environment. By incorporating additional flow statistics or hybrid approaches that combine flow-level and selective packet-level features, the system can achieve higher detection accuracy while maintaining efficiency.

Future work can also focus on developing a **fully integrated risk-based policy engine**, combining threat scores with Quality of Service (QoS) metrics to enable more intelligent and context-aware decision-making. This would align with the overall goal of creating a unified network management framework.

Additionally, optimizing the system for **scalability and deployment in real-world environments**, such as cloud-based infrastructures or large enterprise networks, would increase its practical applicability. Enhancements in API performance, fault tolerance, and distributed processing can further improve reliability.

Overall, these improvements will contribute to building a more **adaptive, robust, and intelligent network security system**.

## REFERENCES

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116.
- [2] A. Thakkar and R. Lohiya, "A Review of the Advancement in Intrusion Detection Datasets," *Procedia Computer Science*, vol. 167, pp. 636–645, 2020.
- [3] Z. K. Maseer, R. Yusof, N. Bahaman, S. A. Mostafa, and C. F. M. Foozy, "Benchmarking of Machine Learning for Anomaly-Based Intrusion Detection Systems," *IEEE Access*, vol. 9, pp. 22351–22370, 2021.
- [4] L. Liu, P. Wang, J. Lin, and L. Liu, "Intrusion Detection of Imbalanced Network Traffic Based on Machine Learning," *IEEE Access*, vol. 9, pp. 7550–7563, 2021.
- [5] X. Gao, C. Shan, C. Hu, Z. Niu, and Z. Liu, "An Adaptive Ensemble Machine Learning Model for Intrusion Detection," *IEEE Access*, vol. 7, pp. 82512–82521, 2019.
- [6] M. A. Siddiqi and W. Pak, "An Agile Approach to Enhance Machine Learning-Based Network Intrusion Detection," *IEEE Access*, vol. 9, pp. 137494–137513, 2021.
- [7] N. McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.
- [8] T. Nadeau and K. Gray, *SDN: Software Defined Networks*. Sebastopol, CA, USA: O'Reilly Media, 2013.
- [9] J. Kreutz et al., "Software-Defined Networking: A Comprehensive Survey," *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, 2015.
- [10] Canadian Institute for Cybersecurity, "CICIDS2017 Dataset," University of New Brunswick, 2017. [Online]. Available: <https://www.unb.ca/cic/datasets/ids-2017.html>
- [11] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [12] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [12] G. Bandarupalli, "Efficient Deep Neural Network for Intrusion Detection Using CIC-IDS2017 Dataset," *IEEE Conference*, 2025. :contentReference[oaicite:11]{index=11}